CSE-434: Socket Project

Hunter Flechsig, 1225292003

Design

a) Message Format

All messages are exchanged as JSON strings. JSON was chosen because it is easily readable, and easy to parse and validate keys in Python

Each message has at two top-level fields:

- "type": the command name (string)
- "data": object containing command parameters

Message Formats:

- Register-user

```
{
  "type": "register-user",
  "data": {
    "user-name": <user-name>,
    "ipv4": <IPv4-addr>,
    "m-port": <m-port>,
    "c-port" <c-port>:
  }
}
```

- register-disk

```
{
  "type": "register-disk",
  "data": {
    "disk-name": <disk-name>,
    "ipv4": <IPv4-addr>,
    "m-port": <m-port>,
    "c-port" <c-port>:
  }
}
```

- configure-dss

```
{
  "type": "configure-dss",
  "data": {
    "dss-name": <dss-name>,
    "n": <n>,
    "Striping-unit": <striping-unit>
  }
}
```

- deregister-user

```
{
  "type": "deregister-user",
  "data": {
    "user-name": <user-name>
  }
}
```

- deregister-disk

```
{
  "type": "deregister-disk",
  "data": {
    "disk-name": <disk-name>
  }
}
```

- **Manager Responses -** Every request receives a JSON response with:

```
{
  "status": "success" | "failure",
  "message": <message>
}
```

Update Format: update message format:

- The different message formats between responses from manager and other processes became hard to manager, so message format was standardized to the following format

{
    action : <command / action >
    type : < request | response >
    error : < error message if error >
    status : < ERROR | SUCCESSFUL >
    data : { < data associated with action >
    id : < id of request >
}

- This message format is used for every message passed between processes
- Id was also added so we can key events and inform threads when it gets back a response of same id in port handling thread

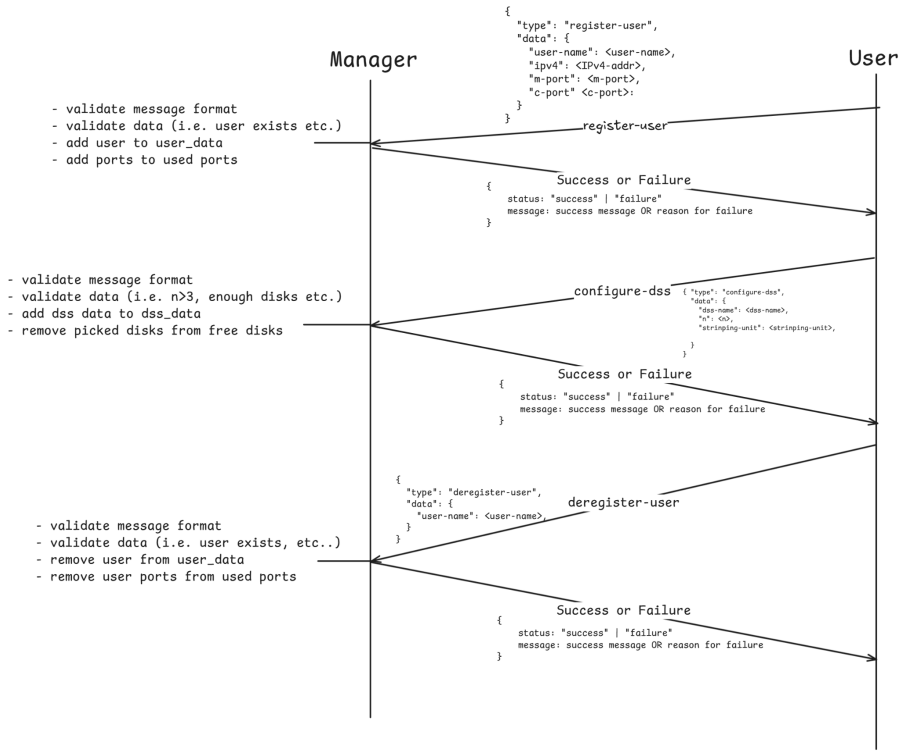
New Messages: all are json simplified to one line. SUCCESS and FAIL simplified to S and F respectively. Request and response simplified to req and res respectively

- { action: register-user, req, S, {name, ip, c_port, m_port}, id }
- { action: register-user, res, S|F, none|error msg, {}, id }
- { action: deregister-user, req, S, {name}, id }
- { action: deregister-user, res, S|F, none|error msg, {name}, id }
- { action: register-disk, req, S, {name, ip, c_port, m_port}, id }
- { action: register-disk, res, S|F, none|error msg, {}, id }
- { action: deregister-disk, req, S, {name}, id }
- { action: deregister-disk, res, S|F , none|error, {name}, id }
- { action: configure-dss, req, S, {name, num_disks, striping_unit}, id }
- { action: configure-dss, res, S|F , none|error, {}, id }
- { action: copy-file, req, S, {owner, file-name, file-size, dss-name}, id }
- { action: copy-file, res, S|F, none|error , {owner, file-name, file-size, dss-name}, id }
- { action: copy-complete, req, S, {}, id }
- { action: copy-complete, res, S|F , none|error, {}, id }
- { action:copy-block, req, S, {file-name, block-index, block-type, stripe-index, num-stripes, num-chunks}, id }
- { action:copy-block, res, S|F, none|error msg, {}, id }
- { action:copy-chunk, req, S, {file-name, stripe-index, chunk-index, chunk-data}, id }
- { action:read-file, req, S, {dss-name, file-name, owner}, id }
- { action:read-file, res, S|F, none|error msg, {file-size, num_disks, striping_unit, disks}, id }
- { action: read-complete, res, S, {dss-name}, id }
- { action: read-complete, res, S|F , none|error, {}, id }
- { action: read-block, req, S, {file-name, stripe-index}, id}
- { action: read-block, res, S|F, none|error msg, {file-name, stripe-index}, id}
- { action: read-chunks, req, S, {file-name, stripe-index, block-index, finish-id}, id}

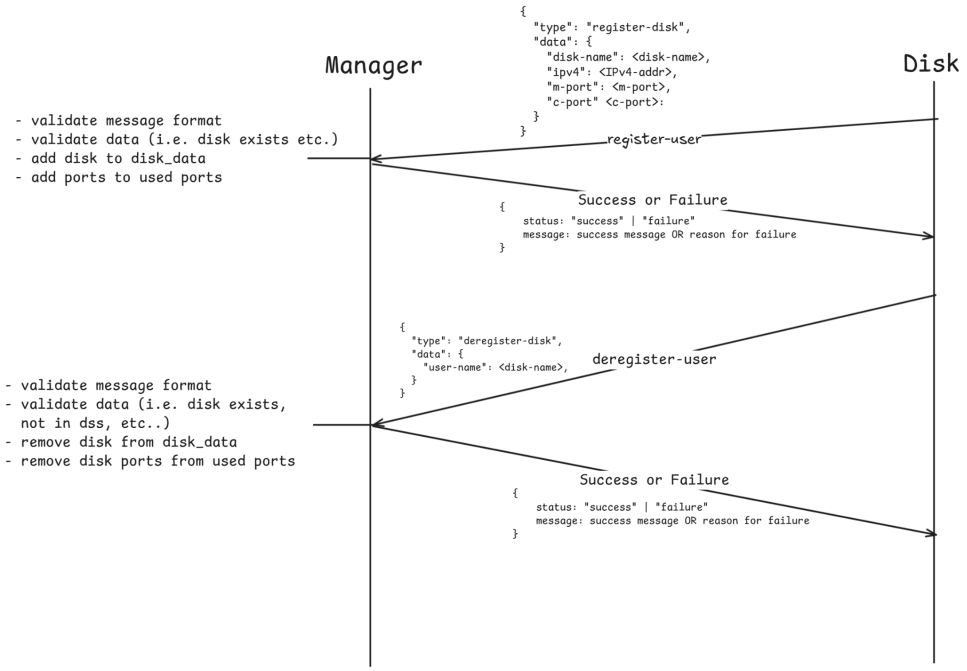
- { action: read-chunks, res, S, {file-name, block-index, chunk-index, chunk_data}, id}
- { action: read-chunk, res, S|F, none|error msg, {file-name, block-index, chunk-index, chunk_data, finish-id}, id}
- { action: disk-failure, req, S, {dss-name}, id}
- { action: disk-failure, res, S|F, none|error msg, {disks, striping_unit, num_disks, files}, id}
- { action: fail, req, S, {}, id}
- { action: fail, res, S|F, none|error msg, {}, id}
- { action: recovery-complete, req, S, {}, id}
- { action: recovery-complete, res, S|F, none|error msg, {}, id}
- { action: ls, req, S, {}, id}
- { action: ls, res, S|F, none|error msg, {n, striping-unit, disks, files}, id}
- { action: decommission-dss, req, S, {dss-name}, id}
- { action: decommission-dss, res, S|F, none|error msg, {dss-name}, id}
- { action: decommission-disk, req, S, {}, id}
- { action: decommission-disk, res, S|F, none|error msg, {}, id}

b) Time Space Diagrams

- Manager-User commands: register-user, configure-dss, deregister-user



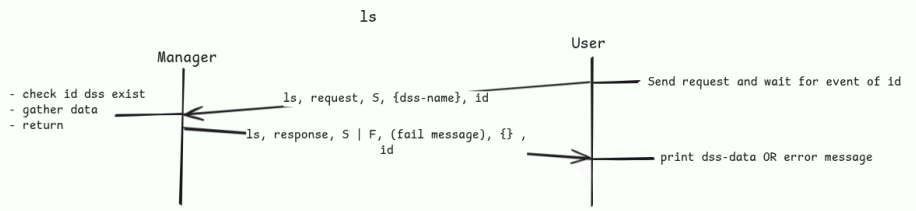
- Manger-Disk commands: register-disk, unregister-disk



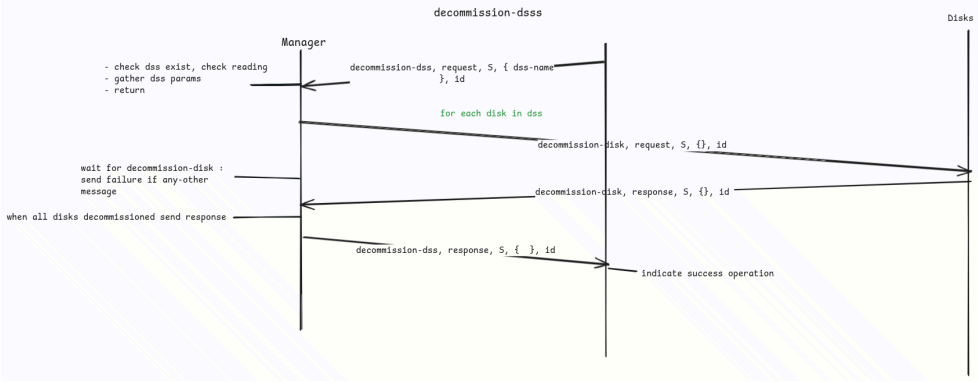
Additions: New Diagrams

- Previous Diagrams are the same but implement a new message format. For example in register user we send a register-user request, of type request, status “SUCCESSFUL”, data { name : <name>}. Manager processes request, then responds with register-user, type response, status of response (“FAILURE” or “SUCCESSFUL”), with an error message if there was an error.
- In diagrams simply message into: action, type, status, (error), data, id

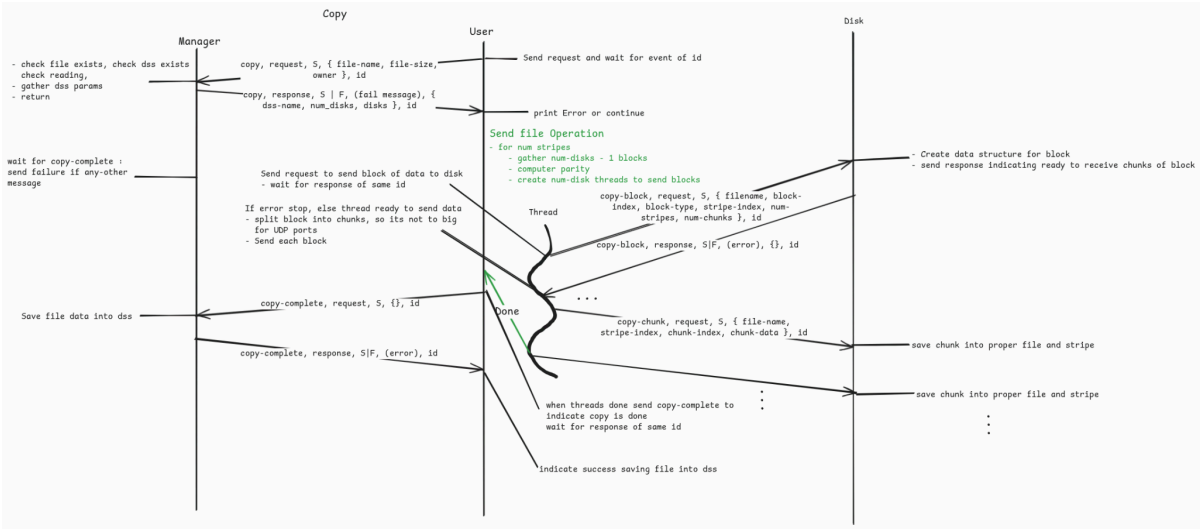
- **ls**



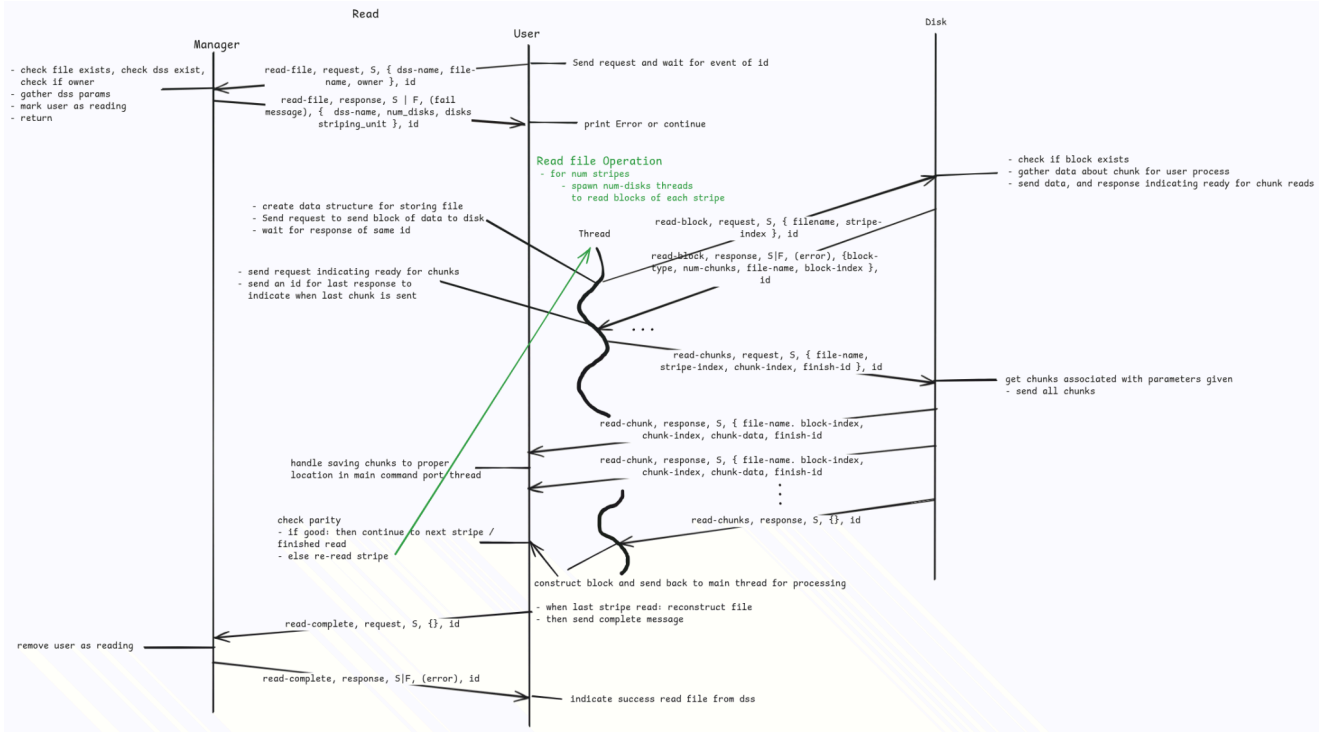
- **decommission-dss**



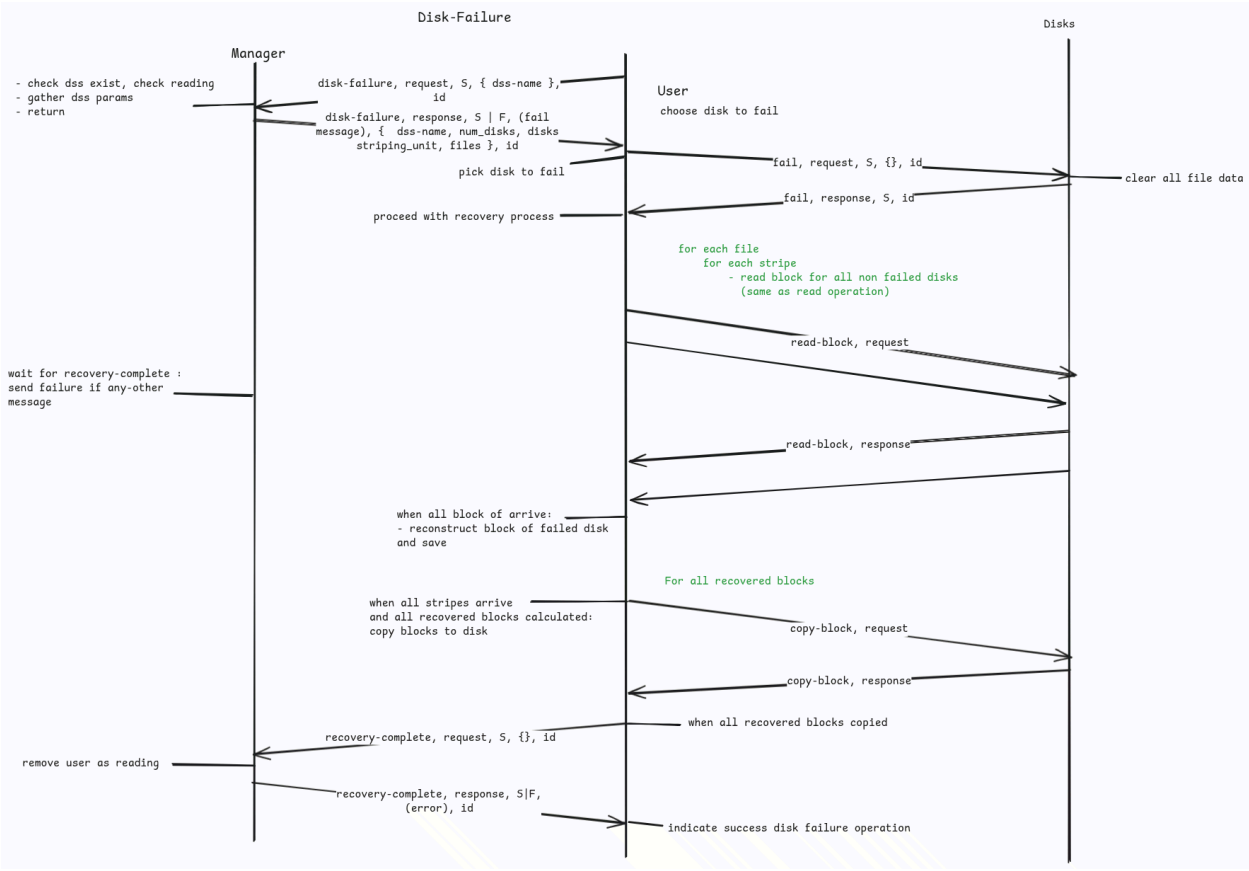
- **copy**


Copy
Manager
User
Disk
- check file exists, check dss exists
check reading,
- gather dss params
- return
copy, request, S, { file-name, file-size, owner }, id
copy, response, S | F, (fail message), { dss-name, num_disks, disks }, id
Send request and wait for event of id
print Error or continue
Send file Operation
- for num stripes
- gather num-disks - 1 blocks
- computer parity
- create num-disk threads to send blocks
wait for copy-complete :
send failure if any-other message
Send request to send block of data to disk
- wait for response of same id
If error stop, else thread ready to send data
- split block into chunks, so its not to big for UDP ports
- Send each block
Thread
copy-block, request, S, { filename, block-index, block-type, stripe-index, num-stripes, num-chunks }, id
- Create data structure for block
- send response indicating ready to receive chunks of block
copy-block, response, S|F, (error), {}, id
Save file data into dss
copy-complete, request, S, {}, id
Done
copy-chunk, request, S, { file-name, stripe-index, chunk-index, chunk-data }, id
save chunk into proper file and stripe
copy-complete, response, S|F, (error), id
when threads done send copy-complete to indicate copy is done
wait for response of same id
indicate success saving file into dss

- **Read**


Read
Manager
User
Disk
- check file exists, check dss exist,
check if owner
- gather dss params
- mark user as reading
- return
read-file, request, S, { dss-name, file-name, owner }, id
read-file, response, S | F, (fail message), { dss-name, num_disks, disks striping_unit }, id
Send request and wait for event of id
print Error or continue
Read file Operation
- for num stripes
- spawn num-disks threads to read blocks of each stripe
- check if block exists
- gather data about chunk for user process
- send data, and response indicating ready for chunk reads
- create data structure for storing file
- Send request to send block of data to disk
- wait for response of same id
Thread
read-block, request, S, { filename, stripe-index }, id
read-block, response, S|F, (error), {block-type, num-chunks, file-name, block-index }, id
- send request indicating ready for chunks
- send an id for last response to indicate when last chunk is sent
read-chunks, request, S, { file-name, stripe-index, chunk-index, finish-id }, id
get chunks associated with parameters given
- send all chunks
read-chunk, response, S, { file-name. block-index, chunk-index, chunk-data, finish-id
handle saving chunks to proper location in main command port thread
read-chunk, response, S, { file-name. block-index, chunk-index, chunk-data, finish-id
read-chunks, response, S, {}, id
check parity
- if good: then continue to next stripe / finished read
- else re-read stripe
construct block and send back to main thread for processing
- when last stripe read: reconstruct file
- then send complete message
read-complete, request, S, {}, id
remove user as reading
read-complete, response, S|F, (error), id
indicate success read file from dss

- **Disk-Failure:** the following is an abstraction: uses same logic as read, and copy for reading and copying blocks of data to and from disks



c) Design Decisions

Data Structures: System uses dictionaries because JSON easily maps to Python dictionaries. This means there is not much processing required to store data from clients. They are also easy to look up since we are enforcing unique names. Using sets for ports and free disks provides fast membership tests to enforce constraints. The separation of data also makes it easier to extend the project when handling additional commands like copy, read, and disk-failure in the future.

In the Manager we maintain 3 dictionaries and 2 sets.

1. user_data dictionary - using user-name as key
    - stores : name, m-port, c-port, and ip of each registered user
    - Purpose: enforce unique usernames, validate registration, track users
2. disk_data dictionary - using disk-name as key
    - stores : name, m-port, c-port, and ip of each registered disk

    - Purpose: enforce unique disk names, track availability of disks
3. dss_data dictionary - using dss-name as key
    - stores: number of disks n, striping size, and list of disks in array
    - **update:** add files key with names of files in dss - key into files_data
    - Purpose: validate new DSS creation, track configuration of arrays
4. used_ports set
    - Track active m and c ports when registering users and disks
    - Purpose: track all m-port and c-port in use to enforce uniqueness
5. free_disks set
    - Track disks not assigned to DSS
    - Purpose: track all disks not yet configured into a DSS for quick selection
6. file_data dict - file name as key
    - track files saved to dss's
    - Store: name, size, owner

New Data Structures:

User:

1. files_data dict - used for buffer data read from dss's
    - Stores at keys [filename][block_index]: chunks (data chunks received from disks) array, block_type, block_index

Disk:

2. files_data dict - used for saving data copied from users
    - Stores at keys [filename][stripe-index]: chunks (data chunks received from disks) array, block_type, block_index, stripe-index

Error Handling and Validation

The system implements robust data validation and error handling:

- Verify that a valid message type is sent to the manager.
- Check that all the necessary fields exist in the message data.
- Validate each field, enforcing constraints on ports, naming, uniqueness, etc.
- If an error occurs, an error response with a failure indicator and a descriptive message is sent back to the client so they understand the issue.

The system uses structured error responses with status indicators (success/failure) and error messages, which helps with debugging and user feedback.

Also, the use of select() with timeouts in the manager prevents blocking

Event Threading System

For the second part of the project, I implemented a custom event-based threading mechanism through the **ThreadEvents** class to manage asynchronous request–response operations across UDP sockets. This allowed for a more sequential flow of operations within threads.

Each request is assigned a unique UUID that correlates with a Python threading.Event object stored in the ThreadEvents registry.
When a process sends a request, it generates a unique ID, creates an event for that ID, sends the message, and then blocks on event.wait() with a configurable timeout.

At the same time, separate listener threads wait for incoming messages on the manager and command ports. When a response arrives, these threads check if the message ID exists in the event registry using event_exists(). If it does, they call set_event() to trigger the waiting thread and store the response data.

The timeout provides error handling for network failures or unresponsive processes. This event-driven architecture cleanly separates the message-receiving threads from the main logic thread.

Decommission-DSS

- Send decommission-dss commands to all disks in the DSS
- Disks clear and send back their responses
- Once all disks are cleared, the DSS is successfully decommissioned
- Operation completes, and a response is sent to the user

Copy File and Read File

Challenges:

- UDP message size limit
- Initially, I tried sending data in the striping unit, but discovered that UDP can only handle packets up to a certain size.

Solution:

- **Sending:** Split data blocks into smaller, receivable chunks.
    - When sending, we also issue a request to notify the receiver that multiple chunks will follow, and wait for a response before continuing to send.
    - This increases the complexity of the receiving logic.
- **Receiving:** Since multiple chunks need to be received, we implement commands to signal that a process is ready to receive these chunks so it can prepare the necessary data structures.

Another design consideration was avoiding the use of spawned threads to process individual chunks, since maintaining order was not guaranteed and the overhead of a call–response for each chunk was not justified.

Instead, the main command port threads receive enough information to read chunks into the appropriate data structure. A final command indicates completion once all chunks have been sent so that any follow-up operations can proceed.

Simulating Disk Failure

This step is relatively straightforward after implementing the read and copy block mechanisms. We reuse the thread responsible for reading blocks to read the necessary data and compute the reconstructed block. Then, we reuse the copying thread to write the reconstructed block back to the disk. No major design changes were required.

Threads

- **Main Thread:** Used by disks and the user for processing commands and performing sequential operations.
- **Command and Manager Threads:** Each disk runs threads to handle incoming messages on these ports, process known commands, or set events to notify the main thread that data is ready.
- **Manager Command Thread:** A dedicated thread in the manager handles user input for closing the program.

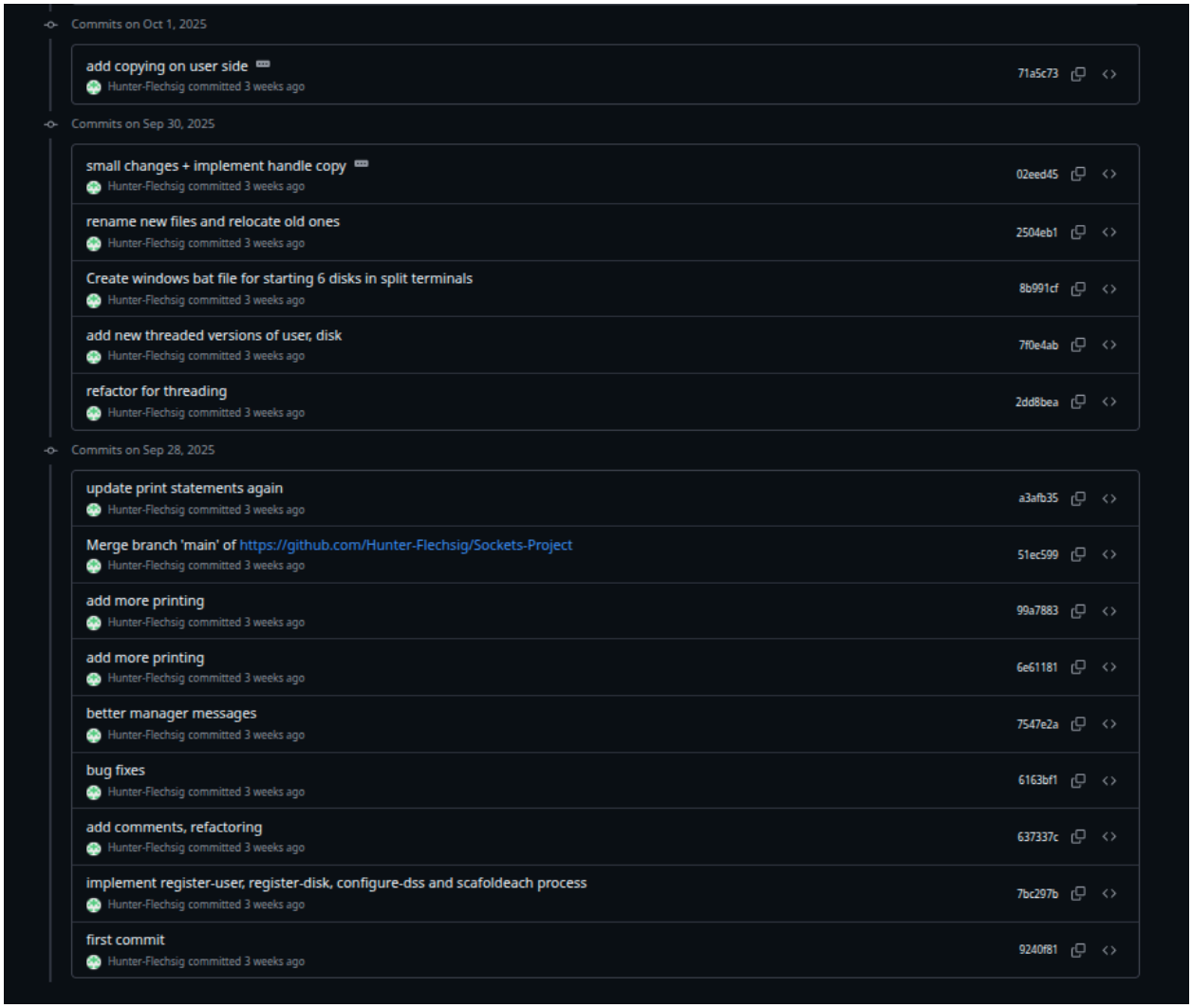
d) GitHub

i.

```
hunter@hunter-Nitro-ANV15-51:~/Coding/computer-networks/Sockets-Project$ git log --pretty=format:"%h - %an, %ad (Commit) - %cd (Author)"
9ef2b46 - Hunyer-Flechsig, Sun Oct 19 14:35:08 2025 -0700 (Commit) - Sun Oct 19 14:35:08 2025 -0700 (Author)
6528c4d - Hunyer-Flechsig, Sun Oct 19 11:33:51 2025 -0700 (Commit) - Sun Oct 19 11:33:51 2025 -0700 (Author)
29a0547 - Hunyer-Flechsig, Sun Oct 19 11:28:32 2025 -0700 (Commit) - Sun Oct 19 11:28:32 2025 -0700 (Author)
60f9b47 - Hunyer-Flechsig, Sun Oct 19 10:23:09 2025 -0700 (Commit) - Sun Oct 19 10:23:09 2025 -0700 (Author)
23c16cb - Hunyer-Flechsig, Sat Oct 18 22:40:14 2025 -0700 (Commit) - Sat Oct 18 22:40:14 2025 -0700 (Author)
1c3c9ba - Hunyer-Flechsig, Sat Oct 18 00:42:59 2025 -0700 (Commit) - Sat Oct 18 00:42:59 2025 -0700 (Author)
ef75561 - Hunyer-Flechsig, Sat Oct 18 00:37:45 2025 -0700 (Commit) - Sat Oct 18 00:37:45 2025 -0700 (Author)
926ca83 - Hunyer-Flechsig, Fri Oct 17 00:48:19 2025 -0700 (Commit) - Fri Oct 17 00:48:19 2025 -0700 (Author)
d54ea3a - Hunyer-Flechsig, Thu Oct 16 23:35:48 2025 -0700 (Commit) - Thu Oct 16 23:35:48 2025 -0700 (Author)
c1227dc - Hunyer-Flechsig, Thu Oct 16 20:45:21 2025 -0700 (Commit) - Thu Oct 16 20:45:21 2025 -0700 (Author)
a2dcfa6 - Hunyer-Flechsig, Wed Oct 15 17:18:04 2025 -0700 (Commit) - Wed Oct 15 17:18:04 2025 -0700 (Author)
3b3200c - Hunyer-Flechsig, Wed Oct 15 17:01:19 2025 -0700 (Commit) - Wed Oct 15 17:01:19 2025 -0700 (Author)
16f4564 - Hunyer-Flechsig, Tue Oct 14 23:08:23 2025 -0700 (Commit) - Tue Oct 14 23:08:23 2025 -0700 (Author)
fd72aee - Hunyer-Flechsig, Tue Oct 14 23:07:25 2025 -0700 (Commit) - Tue Oct 14 23:07:25 2025 -0700 (Author)
0686f69 - Hunyer-Flechsig, Tue Oct 14 21:37:33 2025 -0700 (Commit) - Tue Oct 14 21:37:33 2025 -0700 (Author)
4a87983 - Hunter-Flechsig, Sat Oct 4 15:36:23 2025 -0700 (Commit) - Sat Oct 4 15:36:23 2025 -0700 (Author)
0d1e0fa - Hunter-Flechsig, Fri Oct 3 00:46:36 2025 -0700 (Commit) - Fri Oct 3 00:46:36 2025 -0700 (Author)
be68822 - Hunter-Flechsig, Thu Oct 2 11:38:07 2025 -0700 (Commit) - Thu Oct 2 11:38:07 2025 -0700 (Author)
71a5c73 - Hunter-Flechsig, Wed Oct 1 16:56:51 2025 -0700 (Commit) - Wed Oct 1 16:56:51 2025 -0700 (Author)
02eed45 - Hunter-Flechsig, Tue Sep 30 22:39:59 2025 -0700 (Commit) - Tue Sep 30 22:39:59 2025 -0700 (Author)
2504eb1 - Hunter-Flechsig, Tue Sep 30 22:39:13 2025 -0700 (Commit) - Tue Sep 30 22:39:13 2025 -0700 (Author)
8b991cf - Hunter-Flechsig, Tue Sep 30 22:38:32 2025 -0700 (Commit) - Tue Sep 30 22:38:32 2025 -0700 (Author)
7f0e4ab - Hunter-Flechsig, Tue Sep 30 18:37:21 2025 -0700 (Commit) - Tue Sep 30 18:37:21 2025 -0700 (Author)
2dd8bea - Hunter-Flechsig, Tue Sep 30 00:06:05 2025 -0700 (Commit) - Tue Sep 30 00:06:05 2025 -0700 (Author)
a3afb35 - Hunter-Flechsig, Sun Sep 28 20:06:37 2025 -0700 (Commit) - Sun Sep 28 20:06:37 2025 -0700 (Author)
51ec599 - Hunter-Flechsig, Sun Sep 28 19:47:17 2025 -0700 (Commit) - Sun Sep 28 19:47:17 2025 -0700 (Author)
99a7883 - Hunter-Flechsig, Sun Sep 28 19:44:03 2025 -0700 (Commit) - Sun Sep 28 19:47:05 2025 -0700 (Author)
6e61181 - Hunter-Flechsig, Sun Sep 28 19:44:03 2025 -0700 (Commit) - Sun Sep 28 19:45:46 2025 -0700 (Author)
7547e2a - Hunter-Flechsig, Sun Sep 28 19:05:03 2025 -0700 (Commit) - Sun Sep 28 19:05:03 2025 -0700 (Author)
6163bf1 - Hunter-Flechsig, Sun Sep 28 17:28:59 2025 -0700 (Commit) - Sun Sep 28 17:28:59 2025 -0700 (Author)
637337c - Hunter-Flechsig, Sun Sep 28 14:59:41 2025 -0700 (Commit) - Sun Sep 28 14:59:41 2025 -0700 (Author)
7bc297b - Hunter-Flechsig, Sun Sep 28 12:19:03 2025 -0700 (Commit) - Sun Sep 28 12:19:03 2025 -0700 (Author)
9240f81 - Hunter-Flechsig, Sun Sep 28 10:18:04 2025 -0700 (Commit) - Sun Sep 28 10:18:04 2025 -0700 (Author)
hunter@hunter-Nitro-ANV15-51:~/Coding/computer-networks/Sockets-Project$
```

ii.


Commits
main
All users
All time
Commits on Oct 19, 2025
add choosing dss to copy file to
Hunter-Flechsig committed 2 hours ago
9ef2b46
fix command line args
Hunter-Flechsig committed 5 hours ago
6528c4d
add print statements
Hunter-Flechsig committed 5 hours ago
29a0547
update start disks script
Hunter-Flechsig committed 6 hours ago
60f9b47
Commits on Oct 18, 2025
comments
Hunter-Flechsig committed 18 hours ago
23c16cb
add extra comments to utils
Hunter-Flechsig committed 2 days ago
1c3c9ba
Finished adding print statements and comments
Hunter-Flechsig committed 2 days ago
ef75561
Commits on Oct 17, 2025
first working disk failure recovery
Hunter-Flechsig committed 3 days ago
926ca83
Commits on Oct 16, 2025
read, write working correctly with reconstruction
Hunter-Flechsig committed 3 days ago
d54ea3a
copy working again
Hunter-Flechsig committed 3 days ago
c1227dc
Commits on Oct 15, 2025
read, copy working before refactor
Hunter-Flechsig committed 4 days ago
a2dcfa6
Add disk failure between manager and user and disk
Hunter-Flechsig committed 4 days ago
3b3200c
Commits on Oct 14, 2025
start all
Hunter-Flechsig committed 5 days ago
16f4564
open script
Hunter-Flechsig committed 5 days ago
fd72aee
remove un-needed files
Hunter-Flechsig committed 5 days ago
0686f69
Commits on Oct 4, 2025
copy and read with sparity checking
Hunter-Flechsig committed 2 weeks ago
4a87983
Commits on Oct 3, 2025
read working
Hunter-Flechsig committed 2 weeks ago
0d1e0fa
Commits on Oct 2, 2025
implement saving user blocks in disk
Hunter-Flechsig committed 2 weeks ago
be68822



iii.

```
hunter@hunter-Nitro-ANV15-51:~/Coding/computer-networks/Sockets-Project$ git reflog
9ef2b46 (HEAD -> main, origin/main, origin/HEAD) HEAD@{0}: commit: add choosing dss to copy file to
6528c4d HEAD@{1}: commit: fix command line args
29a0547 HEAD@{2}: commit: add print statements
60f9b47 HEAD@{3}: commit: update start disks script
23c16cb HEAD@{4}: commit: comments
1c3c9ba HEAD@{5}: commit: add extra comments to utils
ef75561 HEAD@{6}: commit: Finished adding print statements and comments
926ca83 HEAD@{7}: commit: first working disk failure recovery
d54ea3a HEAD@{8}: checkout: moving from main to main
d54ea3a HEAD@{9}: commit: read, write working correctly with reconstruction
c1227dc HEAD@{10}: commit: copy working again
a2dcfa6 HEAD@{11}: reset: moving to a2dcfa6fd909bd82e38c4ad16063fa156d6b2865
ed1be84 HEAD@{12}: commit: copy working refactor
a2dcfa6 HEAD@{13}: commit: read, copy working before refactor
3b3200c HEAD@{14}: commit: Add disk failure between manager and user and disk
16f4564 HEAD@{15}: commit: start all
fd72aee HEAD@{16}: commit: open script
0686f69 HEAD@{17}: commit: remove un-needed files
4a87983 HEAD@{18}: clone: from github.com:Hunter-Flechsig/Sockets-Project.git
hunter@hunter-Nitro-ANV15-51:~/Coding/computer-networks/Sockets-Project$
```

iv.

```
hunter@hunter-Nitro-ANV15-51:~/Coding/computer-networks/Sockets-Project$ git fsck
Checking object directories: 100% (256/256), done.
Checking objects: 100% (100/100), done.
dangling blob 3e8a7e0c4672a5260b733c6b0aa44af05b45709d
hunter@hunter-Nitro-ANV15-51:~/Coding/computer-networks/Sockets-Project$
```

e) Videos

Milestone - Youtube Link: https://youtu.be/zTyOlmfR__s

- a) Not applicable
- b) 2 hosts: https://youtu.be/zTyOlmfR__s?t=4
- c) Start manager, 3 disks and 2 users: https://youtu.be/zTyOlmfR__s?t=36
- d) Configure DSS: https://youtu.be/zTyOlmfR__s?t=182
- e) Configure DSS failure: https://youtu.be/zTyOlmfR__s?t=215
- f) Deregister and close processes: https://youtu.be/zTyOlmfR__s?t=251

Full Project Video - Youtube Link: https://youtu.be/JjhMlfKEXKo

- a) Compilation: Not applicable
- b) Demo setup: https://youtu.be/JjhMlfKEXKo?t=8
- c) Start programs: https://youtu.be/JjhMlfKEXKo?t=32
- d) user commands 1: https://youtu.be/JjhMlfKEXKo?t=82
- e) user commands 2: https://youtu.be/JjhMlfKEXKo?t=461
- f) disk-failure: https://youtu.be/JjhMlfKEXKo?t=560
- g) read from the failed disk: https://youtu.be/JjhMlfKEXKo?t=645
- h) decommission and close: https://youtu.be/JjhMlfKEXKo?t=710